

Taller 03 Posix Semáforos

Marlon García

Carrera de Ingeniería de Sistemas

Departamento de Ingeniería de Sistemas

Pontificia Universidad Javeriana

John Jairo Corredor Franco PhD.

19 de Mayo del 2026

Índice

Resumen	3
1. Introducción	4
2. Refactorización de la Solución Multihilo (<code>posixSincro.c</code>)	5
2.1. El Problema de la Finalización Violenta	5
2.2. La Solución: Sincronización Elegante mediante Hilos	5
3. Reingeniería del Sistema Multiproceso (IPC)	7
3.1. Deficiencias del Código Anterior (<code>ola/</code>)	7
3.2. Creación del Soporte Modular (<code>consumer.h</code>)	7
3.3. Patrón de Espera Conectiva (Bucle Paciente)	8
3.4. Gestión Dinámica de Consumo y Parada Segura	9
3.5. Prevención de Fugas en Kernel mediante Manejo de Señales	9
4. Automatización de Compilación (<code>Makefile</code>)	11
5. Estudio Comparativo y Matriz de Robustez	12
6. Conclusiones	13

Resumen

Este informe detalla la evolución técnica y metodológica de un sistema clásico de sincronización productor-consumidor. El punto de partida consistía en un conjunto rudimentario de programas con problemas severos de portabilidad, fugas de recursos del kernel e inconsistencias críticas en la sincronización. Mediante una reingeniería sistemática, se transformó este software heredado (*legacy*) en un sistema robusto, modularizado y tolerante a fallos. Se analiza el paso de una concurrencia multihilo de un único proceso (`posixSincro.c`) a una arquitectura multiproceso real coordinada mediante variables en memoria compartida (`shm`) y semáforos POSIX (`producer.c`, `consumer.c` y `consumer.h`), gestionada de forma limpia por un archivo de automatización de compilación (`Makefile`). Los resultados demuestran la viabilidad de lograr robustez a nivel industrial en código de bajo nivel en sistemas tipo UNIX mediante el control estricto del ciclo de vida de los recursos IPC y la gestión activa de señales del sistema operativo.

1 Introducción

La programación concurrente y de sistemas es uno de los pilares del diseño de software moderno. Sin embargo, trae consigo desafíos inherentes como condiciones de carrera (*race conditions*), bloqueos mutuos (*deadlocks*) y la gestión del estado compartido. El problema del Productor-Consumidor representa el arquetipo clásico para resolver la transmisión de datos asíncrona entre entidades concurrentes.

En entornos UNIX, existen dos enfoques principales para implementar este patrón:

- **Multihilo (Single-Process):** Hilos ligeros que comparten el espacio de direccionamiento virtual del proceso. La comunicación es directa a través de variables globales, y la sincronización se realiza con exclusión mutua (*mutexes*) y variables de condición (*condition variables*).
- **Multiproceso (IPC - Inter-Process Communication):** Procesos independientes con espacios de direccionamiento completamente aislados. La comunicación requiere la intervención del kernel mediante el mapeo de un segmento físico común (Memoria Compartida POSIX, `shm_open/mmap`), coordinado externamente mediante semáforos nominales del sistema (`sem_open`).

El presente documento relata el proceso de refactorización y depuración de ambos modelos para solucionar problemas críticos encontrados en el código original ubicado en la carpeta `ola`, tales como terminaciones violentas de hilos, programas que no compilaban por falta de modularidad y fugas persistentes de recursos de IPC en el sistema operativo.

2 Refactorización de la Solución Multihilo (posixSincro.c)

El programa original ola/posixSincro.c planteaba un modelo multihilo clásico con 10 productores que escribían en un búfer circular de tamaño 5 y un hilo “spooler” (consumidor/impresor) que extraía los datos para imprimirlos. A pesar de su aparente simplicidad, el código presentaba un defecto crítico de diseño en la fase de finalización.

2.1 El Problema de la Finalización Violenta

En la versión heredada (ola/posixSincro.c), el hilo principal esperaba de manera ineficiente y propensa a fallos mediante un ciclo de espera activa (*busy wait*) basado en sleep:

```
1 // C digo original (ola/posixSincro.c)
2 while (lines_to_print) sleep (1);
3 if ((r = pthread_cancel (tid_spooler)) != 0) {
4     fprintf (stderr, "Error = %d (%s)\n", r, strerror (r)); exit (1);
5 }
```

Listing 1: Espera ineficiente y cancelación brusca en el código heredado

Esta estrategia es sumamente peligrosa. El uso de `pthread_cancel` termina bruscamente el hilo `spooler`. Si este hilo se cancela en medio de una sección crítica mientras retiene el mutex `buf_mutex`, el semáforo o recurso queda bloqueado permanentemente, provocando un *deadlock* si otros procesos intentaran acceder a él, además de corromper la salida estándar y provocar fugas de memoria al no liberar las estructuras locales del hilo.

2.2 La Solución: Sincronización Elegante mediante Hilos

Para solucionar esto, se introdujo una variable de estado global protegida llamada `active_producers` que realiza un seguimiento del número de hilos productores activos.

El flujo de finalización ahora se diseña de manera que el hilo `spooler` detecte de forma cooperativa cuándo debe terminar su ejecución. Al finalizar el bucle de producción de cada productor, se decrementa esta variable dentro de la zona protegida por el mutex y se envía una señal a la variable de condición del `spooler`:

```
1 // C digo refactorizado (posixSincro.c)
2 if ((r = pthread_mutex_lock(&buf_mutex)) != 0) { ... }
3 active_producers--;
4 pthread_cond_signal(&spool_cond); // Notificar al spooler
5 if ((r = pthread_mutex_unlock(&buf_mutex)) != 0) { ... }
```

Listing 2: Decremento seguro de productores activos y señalización de salida

Por su parte, el `spooler` ya no corre en un ciclo infinito ciego. Evalúa si quedan líneas por imprimir y si todavía existen productores vivos. Si no hay líneas y los productores han terminado, sale del bucle limpiamente:

```
1 // C digo refactorizado (posixSincro.c)
2 while (!lines_to_print && active_producers > 0)
3     pthread_cond_wait(&spool_cond, &buf_mutex);
4
5 if (!lines_to_print && active_producers == 0) {
6     pthread_mutex_unlock(&buf_mutex);
7     break; // Salida elegante del bucle de impresi n
8 }
```

Listing 3: Condición de salida ordenada en el Spooler

Gracias a esto, el hilo principal puede llamar directamente a `pthread_join` para esperar el fin natural del spooler, asegurando que todos los recursos locales sean liberados por el sistema operativo de forma óptima. Adicionalmente, se agregaron firmas de retorno explícitas (`return NULL;`) en las funciones concurrentes para cumplir de forma estricta con el estándar de POSIX Threads.

3 Reingeniería del Sistema Multiproceso (IPC)

El verdadero desafío de reingeniería residió en los programas multiproceso `producer.c` y `consumer.c`. En el directorio original `ola`, el código presentaba problemas sustanciales que impedían no solo la ejecución robusta, sino incluso la mera compilación del software.

3.1 Deficiencias del Código Anterior (`ola/`)

1. **Inviabilidad de Compilación:** Ninguno de los dos archivos (`producer.c` y `consumer.c`) incluía las cabeceras básicas de biblioteca estándar de C (`<stdio.h>`, `<stdlib.h>`, `<unistd.h>`), ni las de control del sistema de memoria compartida y semáforos POSIX (`<sys/mman.h>`, `<semaphore.h>`). Además, hacían uso del tipo estructurado `compartir_datos` y la constante `BUFFER` sin tenerlos definidos en ningún lugar.
2. **Flujo Estático y Rígido:** El consumidor estaba programado para realizar exactamente 10 ciclos de lectura (`for (int i = 1; i <= 10; i++)`), asumiendo una sincronía perfecta de velocidad 1:1 con el productor. Si el productor enviara más datos o terminara antes, el consumidor se desbordaría o quedaría colgado indefinidamente.
3. **Condición de Carrera en la Inicialización:** Si el consumidor se iniciaba milisegundos antes de que el productor creara y mapeara los objetos de kernel, las llamadas a `sem_open` y `shm_open` en el consumidor fallaban catastróficamente provocando una salida por error (`exit(1)`).
4. **Fugas de Recursos del Kernel:** El sistema no contemplaba la posibilidad de cancelaciones inesperadas (como pulsar `Ctrl+C`). Si un proceso era finalizado por una señal, los semáforos y los archivos compartidos permanecían asignados en el kernel del sistema operativo (habitualmente montados bajo `/dev/shm`), corrompiendo futuras ejecuciones.

3.2 Creación del Soporte Modular (`consumer.h`)

La primera medida correctora consistió en diseñar una interfaz común para centralizar las importaciones del sistema, declarar el tamaño físico del búfer circular y modelar formalmente la memoria de intercambio:

```
1 #ifndef CONSUMER_H
2 #define CONSUMER_H
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <unistd.h>
7 #include <fcntl.h>
8 #include <sys/mman.h>
9 #include <sys/stat.h>
```

```
10 #include <semaphore.h>
11
12 #define BUFFER 5 // Capacidad del buffer circular
13
14 typedef struct {
15     int bus[BUFFER]; // Buffer circular
16     int entrada; // Indice de escritura (productor)
17     int salida; // Indice de lectura (consumidor)
18     int terminado; // Flag de finalizacion
19 } compartir_datos;
20
21 #endif
```

Listing 4: Estructura de datos compartida y constantes de sistema en consumer.h

La adición del flag lógico `terminado` en la estructura de control es el eje que permite que la sincronización deje de ser de tamaño constante y pase a ser de flujo de datos dinámico.

3.3 Patrón de Espera Conectiva (Bucle Paciente)

Para resolver la fragilidad en la puesta en marcha concurrente, se implementó en `consumer.c` un mecanismo de reintentos dinámico (Bucle Paciente). Si al arrancar el consumidor los recursos compartidos aún no han sido instanciados por el productor, el sistema entra en un bucle controlado de hasta 15 intentos, esperando 1 segundo entre ellos, en lugar de abortar abruptamente:

```
1 // Esperar semaforos de manera tolerante a fallos de arranque
2 int retries = 0;
3 while (retries < 15) {
4     vacio = sem_open("/vacio", 0);
5     lleno = sem_open("/lleno", 0);
6     if (vacio != SEM_FAILED && lleno != SEM_FAILED) {
7         break; // Conexion exitosa
8     }
9     if (vacio != SEM_FAILED) { sem_close(vacio); vacio = SEM_FAILED; }
10    if (lleno != SEM_FAILED) { sem_close(lleno); lleno = SEM_FAILED; }
11
12    printf("Esperando al productor (intento %d/15)...\n", retries + 1);
13    sleep(1);
14    retries++;
15 }
```

Listing 5: Bucle paciente de conexion en el Consumidor

Este enfoque elimina la necesidad de controlar manualmente el orden de inicio de los programas en la consola.

3.4 Gestión Dinámica de Consumo y Parada Segura

El bucle principal del consumidor se transformó para admitir el drenaje completo del búfer circular. Incluso cuando el productor marca el canal como finalizado (`terminado = 1`), el consumidor no debe detenerse de inmediato si aún quedan elementos escritos en tránsito (`entrada != salida`):

```
1 while (1) {
2     sem_wait(lleno); // Espera que haya un elemento listo
3
4     % Parada: Productor finalizo Y el buffer se ha vaciado por completo
5     if (compartir->terminado && compartir->entrada == compartir->salida) {
6         printf("[Consumidor] Productor finalizado y buffer vacio. Saliendo
7         ...\\n");
8         break;
9     }
10
11     int item = compartir->bus[compartir->salida];
12     printf("Consumidor: Consume %d\\n", item);
13     compartir->salida = (compartir->salida + 1) % BUFFER;
14
15     sem_post(vacio); % Notificar ranura vacia
16     sleep(2);
17 }
```

Listing 6: Algoritmo de drenaje y parada en consumer.c

3.5 Prevención de Fugas en Kernel mediante Manejo de Señales

Para solventar el grave problema de la acumulación de archivos huérfanos en `/dev/shm` al abortar el programa mediante el teclado (`Ctrl+C`), se instaló un capturador de señales del sistema operativo (`SIGINT` y `SIGTERM`) en ambos ejecutables:

```
1 void limpiar_recursos(int sig) {
2     (void)sig;
3     printf("\\n[Sistema] Interrumpido de forma externa. Limpiando IPC...\\n"
4     );
5     sem_unlink("/vacio");
6     sem_unlink("/lleno");
7     shm_unlink("/memoria_compartida");
8     exit(EXIT_SUCCESS);
9 }
10
11 // En el main():
12 signal(SIGINT, limpiar_recursos);
13 signal(SIGTERM, limpiar_recursos);
```

Listing 7: Manejador de senales para desvincular recursos IPC

Este desarrollo garantiza que el kernel de Linux no acumule descriptores bloqueados y que el sistema comience en un estado completamente limpio en cada inicio, ya que

el productor realiza una limpieza preventiva (llamando a `shm_unlink` y `sem_unlink`) al instanciarse y el consumidor ejecuta la desvinculación definitiva en su salida estándar exitosa.

4 Automatización de Compilación (Makefile)

El código heredado requería largas y propensas a errores líneas de compilación manual en la consola. Con la introducción de componentes que hacen uso de memoria compartida y flujos multihilo, es obligatorio enlazar las bibliotecas del kernel correctas. Para ello se estructuró un Makefile profesional que automatiza el proceso.

El archivo define reglas genéricas y enlaza explícitamente el soporte de hilos POSIX (-pthread) y la biblioteca de extensiones en tiempo real de POSIX (-lrt), requerida en Linux para el correcto mapeo de memoria compartida:

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -pthread
3 LDFLAGS = -pthread -lrt
4
5 TARGETS = posixSincro producer consumer
6
7 all: $(TARGETS)
8
9 posixSincro: posixSincro.c
10     $(CC) $(CFLAGS) $< -o $@
11
12 producer: producer.c consumer.h
13     $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
14
15 consumer: consumer.c consumer.h
16     $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
17
18 clean:
19     rm -f $(TARGETS)
20
21 .PHONY: all clean
```

Listing 8: Makefile automatizado para el entorno concurrente

5 Estudio Comparativo y Matriz de Robustez

A continuación, se resume de manera analítica el impacto cualitativo de la reingeniería aplicada sobre la base de código.

Tabla 1: Matriz Comparativa entre el Código Original (ola/) y el Código Refactorizado

Característica	Código Heredado (Carpeta ola/)	Código Refactorizado (Actual)
Compilación	Fallida. Ausencia total de cabeceras estándar e IPC. Sin Makefile.	Compila al 100% de manera limpia mediante gcc y Makefile estructurado.
Finalización en Hilos	Violenta. <code>pthread_cancel</code> incontrolado. Alto riesgo de deadlock y fuga de locks.	Sincronizada y segura. Uso de banderas compartidas y <code>pthread_join</code> controlado.
Modularidad	Nula. Variables declaradas <i>ad-hoc</i> con código duplicado e incompleto.	Alta. Estructuras comunes encapsuladas en <code>consumer.h</code> .
Inicio Concurrente	Frágil. Orden de arranque mandatorio. Si el consumidor iniciaba primero, abortaba por fallo.	Sincronizado automáticamente. El consumidor reintenta en un bucle paciente y espera al productor.
Ciclo de Producción	Rígido (10 elementos fijos). Imposibilidad de adaptarse a flujos variables.	Dinámico. Parada inteligente condicionada a la bandera <code>terminado</code> y vaciado del búfer.
Persistencia IPC	Crítica. Fugas crónicas de descriptores de semáforos y memoria en el kernel de Linux.	Segura. Gestión de señales de interrupción que desvinculan los recursos del OS en tiempo real.

6 Conclusiones

1. **La Sincronización Cooperativa es Mandatoria:** La cancelación brusca de flujos concurrentes (hilos o procesos) mediante el sistema operativo (`pthread_cancel` o señales sin interceptar) introduce riesgos extremos de inconsistencia y exclusión mutua residual. El diseño correcto debe orientarse a un modelo de finalización cooperativo fundamentado en flags lógicos y sincronización ordenada (`pthread_join`).
2. **La Robustez exige Tolerancia al Arranque Asíncrono:** En sistemas distribuidos o multiproceso reales, no se puede predecir el orden de arranque de las partes. Implementar buffers pacientes de espera evita fallos prematuros de inicialización y confiere autonomía operacional a los binarios concurrentes.
3. **El Ciclo de Vida de los Recursos de Kernel:** Los recursos compartidos creados mediante llamadas de IPC de POSIX tienen persistencia a nivel de kernel, no a nivel de proceso. Esto implica que un programa de bajo nivel tiene la responsabilidad deontológica de interceptar señales de apagado del sistema para dejar el espacio del kernel libre de polución, garantizando la resiliencia del sistema operativo anfitrión.